

Django Templates & Template Language

Dynamic HTML creation, template syntax, variable interpolation, context mapping, template inheritance, debugging, and testing.

1. What is a Django Template?

A **Django Template** is a text file (typically HTML) that defines the presentation layer of a web page. It separates design and layout from business logic, incorporating **Django Template Language (DTL)** constructs to render dynamic data passed from Django views.

2. Django Template Language (DTL) Syntax

Variables `{{ var }}`

Interpolates dynamic values passed from the view's context dictionary into the HTML output.

Tags `{% tag %}`

Controls rendering logic such as loops (`for`), conditionals (`if`), or template inheritance.

Filters `{{ val|filter }}`

Transforms or formats variable values before displaying them (e.g., lower, date, length).

3. Rendering Dynamic Context in Views

Views send dynamic data to templates by passing a Python dictionary as the **context** parameter to the `render()` function.

```
# views.py
from django.shortcuts import render
from .models import MenuItem

def menu_view(request):
    items = MenuItem.objects.all()
    context = {'menu_items': items, 'restaurant_name': 'Little Lemon'}
    return render(request, 'menu.html', context)
```

4. Common DTL Control Flow Constructs

Construct	DTL Code Example	Description
For Loop	<code>{% for item in menu_items %} {{ item.name }} {% empty %} No items. {% endfor %}</code>	Iterates over list/QuerySet. Optional <code>{% empty %}</code> block renders if list is empty.
If Condition	<code>{% if item.price < 10 %} Special! {% else %} Regular {% endif %}</code>	Evaluates boolean logic and outputs content accordingly.
Filters	<code>{{ restaurant_name upper }}</code> / <code>{{ items length }}</code>	Modifies output formatting inline using the pipe (<code> </code>) operator.
Comments	<code>{# Hidden comment #}</code> or <code>{% comment %} ... {% endcomment %}</code>	Excludes template comments from the rendered HTML sent to the client.

5. Template Inheritance

Template inheritance allows developers to build a base skeleton template containing common elements (header, footer, navigation bar) and define overridable **blocks** for child templates.

Base Template (`base.html`)

Defines overall HTML layout and structural blocks using `{% block block_name %}` tags.

Child Template (`menu.html`)

Uses `{% extends "base.html" %}` at the very top line and overrides individual content blocks.

Little Lemon Restaurant

```
{% block content %}{% endblock %}
```

© 2026 Little Lemon

```
{% extends 'base.html' %} {% block title %}Menu - Little Lemon{% endblock %} {% block content %}
```

Our Special Menu

```
    {% for item in menu_items %}

    • {{ item.title }} - ${ item.price }}

    {% endfor %}

{% endblock %}
```

7. Debugging Django Applications

Effective debugging strategies help identify issues in view logic, database queries, and template rendering errors quickly.

Django Debug Mode (`DEBUG = True`)

In development, setting `DEBUG = True` in `settings.py` displays detailed error tracebacks in the browser when exceptions occur.

Warning: Always set `DEBUG = False` in production!

Python Debugger Tool (`breakpoint()`)

Insert `breakpoint()` or `import pdb; pdb.set_trace()` inside view functions to pause execution and inspect variable states directly from the console terminal.

8. Automated Testing in Django

Django's test framework is built on Python's native `unittest` library. It uses a temporary database to ensure tests run cleanly in isolation without affecting real application data.

```
# myapp/tests.py
from django.test import TestCase
from django.urls import reverse
from .models import MenuItem

class MenuItemTest(TestCase):
    def setUp(self):
        # Prepare test data before each test execution
        self.item = MenuItem.objects.create(title="Ice Cream", price=4.50)

    def test_item_creation(self):
        # Verify model field logic
        self.assertEqual(str(self.item), "Ice Cream")
        self.assertEqual(self.item.price, 4.50)

    def test_menu_view_status_code(self):
        # Test HTTP response code using Django Test Client
        client = Client()
        response = client.get(reverse('menu_list'))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Ice Cream")
```

9. Running Django Test Suites

To execute all test cases across installed applications, run the standard CLI command:

```
# Run all tests in the project
python manage.py test

# Run tests for a specific app
python manage.py test myapp
```

Key Testing Takeaways

- `TestCase` automatically creates a clean, temporary database for test runs and destroys it when finished.
- Use `self.assertContains(response, text)` to check if text renders properly inside a dynamic HTML template.
- Use `self.assertEqual()` and `self.assertTrue()` for standard state assertions.